SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 15

Name of Student : Sharwan Uddemari
Enrollment No. : 2303A51957
Batch No. 22

Task 1 – Student Attendance API
Create a RESTful API for student attendance tracking.
Instructions:
- **Endpoints required: o GET /attendance → View attendance records o POST /attendance → Mark attendance o PUT /attendance/{id} → Update attendance record o DELETE /attendance/{id} → Remove attendance entry • Ensure JSON-based request and response handling.**
  Expected Output:
- **API for maintaining and updating attendance records.**

Prompt:-
Create a RESTful API using Node.js and Express for a Student Attendance System. I need four endpoints: GET /attendance (to view all records), POST /attendance (to mark attendance with studentName, date, and status), PUT /attendance/:id (to update a record), and DELETE /attendance/:id (to remove one). Use a simple in-memory array to store data for now. Include basic error handling and comments for documentation. Ensure responses are in JSON format.

Code:-

```
ment15 > JS Task-1.js > ...
// Create a RESTful API using Node.js and Express for a Student Attendance System. I need four endpoint
const express = require('express');
const app = express();
const PORT = 3000;
app.use(express.json());

// In-memory array to store attendance records
let attendanceRecords = [
    // Example record
    {
        id: 1,
        studentName: 'John Doe',
        date: '2023-10-01',
        status: 'present'
    },
    {
        id: 2,
        studentName: 'Jane Smith',
        date: '2023-10-01',
        status: 'absent'
    }
];
let currentId = 2; // To keep track of the next ID to assign
// GET /attendance - View all attendance records
app.get('/attendance', (req, res) => {
    res.json(attendanceRecords);
});
// POST /attendance - Mark attendance
app.post('/attendance', (req, res) => {
    const { studentName, date, status } = req.body;
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    const newRecord = {
        id: currentId++,
        studentName,
        date,
        status
    };
    attendanceRecords.push(newRecord);
    res.status(201).json(newRecord);
});
```

```
// PUT /attendance/:id - Update a record
app.put('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const { studentName, date, status } = req.body;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    attendanceRecords[recordIndex] = { id: parseInt(id), studentName, date, status };
    res.json(attendanceRecords[recordIndex]);
});
// DELETE /attendance/:id - Remove a record
app.delete('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    const deletedRecord = attendanceRecords.splice(recordIndex, 1);
    res.json(deletedRecord[0]);
});
// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on port http://localhost:${PORT}`);
});
```

Output:-

Task – Task:

Pretty-print


```
[
  {
    "id": 1,
    "studentName": "John Doe",
    "date": "2023-10-01",
    "status": "present"
  },
  {
    "id": 2,
    "studentName": "Jane Smith",
    "date": "2023-10-01",
    "status": "absent"
  }
]
```


Justification:-
This code sets up a simple RESTful API for managing student attendance records using Node.js and Express. It includes endpoints for viewing all records, adding new records, updating existing records, and deleting records. The data is stored in an in-memory array, which is suitable for testing and development purposes. Basic error handling is implemented to ensure that required fields are provided and that operations are performed on existing records. All responses are returned in JSON format for consistency and ease of use with front-end applications.

2 Inventory Management API

Develop a REST API for warehouse inventory management.
Instructions:
- **Endpoints required: o GET /inventory → List all items o POST /inventory → Add new item o PUT /inventory/{id} → Update stock quantity o DELETE /inventory/{id} → Remove an item • Implement validation for stock values. Expected Output:**
- **API capable of managing inventory stock levels.**

Prompt:-
//create a RESTful API using Node.js and Express for Warehouse Inventory Management that stores items with an ID, name, and stock quantity in an in-memory array. Provide a GET /inventory endpoint to list all items, a POST /inventory endpoint to add new items, a PUT /inventory/:id endpoint to update the stock quantity, and a DELETE /inventory/:id endpoint to remove items. Include a validation check to ensure that stock quantity cannot be updated to a negative number and return a 400 error if it is.

Code:-

SR UNIVERSITY

```
//create a RESTful API using Node.js and Express for Warehouse Inventory Management that stores items with an ID, name, and stock q
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let inventory = [
    // Sample item for testing
    { id: 1, name: 'Item A', stock: 10 },
    { id: 2, name: 'Item B', stock: 20 },
    { id: 3, name: 'Item C', stock: 30 }
];
let nextId = 4;

// GET /inventory - List all items
app.get('/inventory', (req, res) => {
    res.json(inventory);
});
// POST /inventory - Add new item
app.post('/inventory', (req, res) => {
    const { name, stock } = req.body;
    if (typeof name !== 'string' || typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Name must be a string and stock must be a non-negative number.' });
    }
    const newItem = { id: nextId++, name, stock };
    inventory.push(newItem);
    res.status(201).json(newItem);
}
);
// PUT /inventory/:id - Update stock quantity
app.put('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { stock } = req.body;
    if (typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Stock must be a non-negative number.' });
    }
    const item = inventory.find(i => i.id === id);
    if (!item) {
        return res.status(404).json({ error: 'Item not found.' });
    }
```

```
        item.stock = stock;
        res.json(item);
    });
    // DELETE /inventory/:id - Remove item
    app.delete('/inventory/:id', (req, res) => {
        const id = parseInt(req.params.id);
        const index = inventory.findIndex(i => i.id === id);
        if (index === -1) {
            return res.status(404).json({ error: 'Item not found.' });
        }
        const removedItem = inventory.splice(index, 1);
        res.json(removedItem[0]);
    });
    // Start the server
    const PORT = 3000;
    app.listen(PORT, () => {
        console.log(`Server is running on port http://localhost:${PORT}`);
    });
```

Task – Task:

Output:-


```
Pretty-print
[
  {
    "id": 1,
    "name": "Item A",
    "stock": 10
  },
  {
    "id": 2,
    "name": "Item B",
    "stock": 20
  },
  {
    "id": 3,
    "name": "Item C",
    "stock": 30
  }
]
```


Justification:
The code implements a RESTful API for Warehouse Inventory Management using Node.js and Express. It provides endpoints for listing all items, adding new items, updating stock quantity, and removing items. The code includes validation checks to ensure that stock quantity cannot be negative and returns appropriate error messages for invalid input or when an item is not found.

3 Hotel Room Booking API

Develop a RESTful API for hotel room booking.
Instructions:
- **Endpoints required: o GET /rooms → View available rooms o POST /rooms/book → Book a room o GET /rooms/{id} → View booking details o DELETE /rooms/{id} → Cancel booking • Implement error handling for unavailable rooms. Expected Output:**
- **API simulating hotel room booking operations.**

Prompt:-
Create a RESTful API for a Hotel Room Booking system using Node.js and Express that manages an array of rooms containing an ID, room number, and booking status. Develop a GET /rooms endpoint that returns only available rooms, a POST /rooms/book endpoint to book a specific room, a GET /rooms/:id endpoint to view specific booking details, and a DELETE /rooms/:id endpoint to cancel a booking. Implement error handling to return a message if a user tries to book a room that is already occupied or does not exist.

Code:-

ment15 > JS Task-3.js > ...

```
//Create a RESTful API for a Hotel Room Booking system using Node.js and Exp
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let rooms = [
    // Sample rooms for testing
    { id: 1, roomNumber: '101', isBooked: false },
    { id: 2, roomNumber: '102', isBooked: false },
    { id: 3, roomNumber: '103', isBooked: false }
];
let nextId = 4;
// GET /rooms - List available rooms
app.get('/rooms', (req, res) => {
    const availableRooms = rooms.filter(room => !room.isBooked);
    res.json(availableRooms);
});
// POST /rooms/book - Book a specific room
app.post('/rooms/book', (req, res) => {
    const { roomNumber } = req.body;
    const room = rooms.find(r => r.roomNumber === roomNumber);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (room.isBooked) {
        return res.status(400).json({ error: 'Room is already booked.' });
    }
    room.isBooked = true;
    res.json({ message: `Room ${roomNumber} has been booked.`, room });
});
```

Task – Task:

```
// GET /rooms/:id - View specific booking details
app.get('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    res.json(room);
});
// DELETE /rooms/:id - Cancel a booking
app.delete('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (!room.isBooked) {
        return res.status(400).json({ error: 'Room is not booked.' });
    }
    room.isBooked = false;
    res.json({ message: `Booking for room ${room.roomNumber} has been canceled.`, room });
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-

```
Pretty-print ✔

[
  {
    "id": 1,
    "roomNumber": "101",
    "isBooked": false
  },
  {
    "id": 2,
    "roomNumber": "102",
    "isBooked": false
  },
  {
    "id": 3,
    "roomNumber": "103",
    "isBooked": false
  }
]
```

Justification:

This code implements a RESTful API for a Hotel Room Booking system using Node.js and Express. It provides endpoints to view available rooms, book a specific room, view booking details, and cancel a booking. The API includes error handling to ensure that users cannot

book already occupied rooms or cancel bookings for rooms that are not currently booked and returns appropriate JSON responses for each operation.

4 Online Voting API

Create a RESTful API for an online voting system.
Instructions:

- **Endpoints required: o GET /candidates → List candidates o POST /vote → Cast a vote o GET /results → View voting results**
- **Ensure one vote per user using in-memory validation.**
  Expected Output:
- **API simulating a secure online voting process.**

Prompt:-
Develop a RESTful API for an Online Voting System using Node.js and Express that manages a list of candidates and tracks user participation to ensure one vote per person. Include a GET /candidates endpoint to list all candidates, a POST /vote endpoint that accepts a userId and candidateId while checking an in-memory list to prevent duplicate voting, and a GET /results endpoint to display the current vote totals. If a user attempts to vote a second time, the API should return a message stating they have already voted.

Code:-

SR UNIVERSITY

Task – Task:

```
//Develop a RESTful API for an online voting system using Node.js and Express that manages a lis
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let candidates = [
    // Sample candidates for testing
    { id: 1, name: 'Candidate A', votes: 0 },
    { id: 2, name: 'Candidate B', votes: 0 },
    { id: 3, name: 'Candidate C', votes: 0 }
];
let votes = []; // To track user votes
// GET /candidates - List all candidates
app.get('/candidates', (req, res) => {
    res.json(candidates);
});
// POST /vote - Cast a vote
app.post('/vote', (req, res) => {
    const { userId, candidateId } = req.body;
    if (!userId || !candidateId) {
        return res.status(400).json({ error: 'User ID and Candidate ID are required.' });
    }
    if (votes.includes(userId)) {
        return res.status(400).json({ error: 'You have already voted.' });
    }
    const candidate = candidates.find(c => c.id === candidateId);
    if (!candidate) {
        return res.status(404).json({ error: 'Candidate not found.' });
    }
    candidate.votes += 1;
    votes.push(userId);
    res.json({ message: `Vote cast for ${candidate.name}.`, candidate });
});
// GET /results - Display current vote totals
app.get('/results', (req, res) => {
    res.json(candidates);
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-

```
Pretty-print
[
  {
    "id": 1,
    "name": "Candidate A",
    "votes": 0
  },
  {
    "id": 2,
    "name": "Candidate B",
    "votes": 0
  },
  {
    "id": 3,
    "name": "Candidate C",
    "votes": 0
  }
]
```


Justification:
This code implements a RESTful API for an Online Voting System using Node.js and Express. It provides endpoints to list candidates, cast votes while ensuring one vote per user, and display current vote totals. The API includes error handling to prevent duplicate voting and returns appropriate JSON responses for each operation.

Task 5 – Teacher Management API Task:
Develop a RESTful API to manage teacher records.
Instructions:
• Endpoints required:
o GET /teachers → List all teachers o POST /teachers → Add a new teacher o PUT /teachers/{id} → Update teacher details o DELETE /teachers/{id} → Remove a teacher • Ensure proper error handling and JSON responses. Expected Output:
• Functional API for managing teacher information.

Prompt:-
Build a RESTful API for Teacher Management using Node.js and Express that allows for full CRUD operations on teacher records consisting of ID, name, subject, and department. Implement a GET /teachers endpoint to view the list, a POST /teachers endpoint to add a new record, a PUT /teachers/:id endpoint to update teacher details, and a DELETE /teachers/:id endpoint to remove a record. Ensure the API returns proper JSON responses and 404 error messages if a requested teacher ID is not found in the system.

Code:-

```
});
// DELETE /teachers/:id - Remove a teacher
app.delete('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = teachers.findIndex(t => t.id === id);
    if (index === -1) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    teachers.splice(index, 1);
    res.json({ message: 'Teacher removed successfully.' });
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

```
//Build a RESTful API for Teacher Management using Node.js and Express that allows for full CRUD
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let teachers = [
    // Sample teachers for testing
    { id: 1, name: 'John Doe', subject: 'Mathematics', department: 'Science' },
    { id: 2, name: 'Jane Smith', subject: 'English', department: 'Arts' },
    { id: 3, name: 'Emily Johnson', subject: 'History', department: 'Social Studies' }
];
let nextId = 4;
// GET /teachers - List all teachers
app.get('/teachers', (req, res) => {
    res.json(teachers);
});
// POST /teachers - Add new teacher
app.post('/teachers', (req, res) => {
    const { name, subject, department } = req.body;
    if (!name || !subject || !department) {
        return res.status(400).json({ error: 'Name, subject, and department are required.' });
    }
    const newTeacher = { id: nextId++, name, subject, department };
    teachers.push(newTeacher);
    res.status(201).json(newTeacher);
});
// PUT /teachers/:id - Update teacher details
app.put('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { name, subject, department } = req.body;
    const teacher = teachers.find(t => t.id === id);
    if (!teacher) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    if (name) teacher.name = name;
    if (subject) teacher.subject = subject;
    if (department) teacher.department = department;
    res.json(teacher);
});
// DELETE /teachers/:id - Remove a teacher
```

Output:-

Pretty-print ✓


```
[
  {
    "id": 1,
    "name": "John Doe",
    "subject": "Mathematics",
    "department": "Science"
  },
  {
    "id": 2,
    "name": "Jane Smith",
    "subject": "English",
    "department": "Arts"
  },
  {
    "id": 3,
    "name": "Emily Johnson",
    "subject": "History",
    "department": "Social Studies"
  }
]
```


Justification:
This code implements a RESTful API for Teacher Management using Node.js and Express. It provides endpoints to view the list of teachers, add new teacher records, update existing teacher details, and remove teacher records. The API includes error handling to ensure that required fields are provided when adding or updating a teacher and returns appropriate JSON responses for each operation, including 404 errors when a requested teacher ID is not found in the system.